# OCEAN

## Deployment Guide for Multi-Node Clusters

QEMU Implementation

*Version 1.0*

# 1. Document Overview

This document provides comprehensive deployment instructions for the OCEAN (Open-source CXL Emulation at Hyperscale Architecture and Networking) CXL emulator on a cluster of 16-20 nodes. The guide is specifically tailored for system managers who will be deploying the emulator infrastructure to provide CXL emulation services to users and benchmark application performance.

## 1.1 About OCEAN

OCEAN is a comprehensive emulation framework that enables full CXL 3.0 functionality, including multi-host memory sharing and pooling support. It provides emulation of CXL 3.0 features such as fabric management, dynamic memory allocation, and coherent memory sharing across multiple hosts in advance of real hardware availability.

## 1.2 Key Features

- Complete CXL 3.0 specification implementation
- Memory Device emulation with up to 32 Host-managed Device Memory (HDM) decoders
- Distributed Fabric Manager implementation
- Performance within approximately 3x of projected native CXL 3.0 speeds

## 1.3 Deployment Scope

This guide covers deployment on 16-20 physical nodes using the QEMU-based implementation. Each node will host virtual machines that emulate CXL-enabled systems, connected through a fabric manager that coordinates memory pooling and sharing.

# 2. Prerequisites and System Requirements

## 2.1 Hardware Requirements

| Component | Requirement |
|---|---|
| CPU | x86_64 processor with virtualization support, minimum 4 cores recommended per vm |
| Memory | Minimum 4GB RAM per vm |
| Storage | Minimum 30GB available disk space per vm |
| Network | 10 Gigabit Ethernet or higher, low-latency interconnect preferred for optimal performance |

## 2.2 Software Requirements

- Operating System: Ubuntu 24.04 LTS or compatible Linux distribution
- QEMU (will be built from source as part of deployment)
- CMake 3.10 or higher
- GCC or Clang compiler with C++11 support
- Python 3.8 or higher
- Network utilities: bridge-utils, net-tools, iproute2

## 2.3 Network Configuration

Ensure all nodes are on the same subnet and can communicate freely. You will need a contiguous block of IP addresses for the virtual machines. Plan for at least 2-3 IP addresses per physical node.

## 2.4 Access Requirements

- Root or sudo access on all nodes
- SSH access between all nodes (passwordless SSH recommended)
- Internet connectivity for downloading dependencies and VM images

# 3. Installation and Configuration

## 3.1 Initial Setup on All Nodes

Perform the following steps on each of the 16-20 nodes in your cluster.

### **Step 1:** Clone the OCEAN Repository

On each node, clone the OCEAN repository from GitHub:

```
git clone https://github.com/cxl-emu/OCEAN.git
cd OCEAN
```

### **Step 2:** Run Host Setup Script

Execute the host setup script to install dependencies and configure the system:

```
bash ./script/setup_host.sh
```

This script will install necessary packages including build tools, QEMU dependencies, and network utilities. It may take several minutes to complete.

### **Step 3:** Configure Network Topology

You need 16-20 CXL nodes (virtual machines). These can be deployed in two ways:

- **Option A**: All 16-20 VMs on a single powerful physical server
- **Option B**: VMs distributed across multiple physical servers

Choose the appropriate setup script based on your infrastructure.

#### *Option A: All VMs on Single Physical Host*

Use this if you have one powerful server with sufficient resources.

**Command to run:**

```
cd /OCEAN
# For n CXL nodes (VMs) on this single hostbash
./script/setup_network.sh n
```

### What this does:

- Creates local bridge br0 with IP 192.168.100.1/24
- Creates TAP interfaces: tap0, tap1, tap2, etc. (one per VM)
- All VMs communicate via the local bridge (fast, low latency)

### *Option B: VMs Distributed Across Multiple Physical Hosts*

Use this if you have multiple physical servers and want to distribute the VMs.

**Example Distributions:**

- 4 hosts with 4 VMs each = 16 total VMs
- 5 hosts with 4 VMs each = 20 total VMs
- 2 hosts with 8+8 VMs = 16 total VMs
- 3 hosts with 6+7+7 VMs = 20 total VMs

**1/ Configure Network Interface First:**

Before running the script, edit it to specify your network interface:

```
vi /opt/OCEAN/script/setup_optional_cross_machine_network.sh
```

Find this line and change it:

```
DEV=enp1s0    # CHANGE THIS to your actual network interface
```

To find your interface name, run:

```
ip link show
```

**2/ Setup on Each Physical Host:**

```
cd /OCEAN
# Run this on each host, replacing <num_vms> and <br_ip_suffix>:
bash ./script/setup_optional_cross_machine_network.sh <num_vms> <br_ip_suffix>
```

**Parameters:**

- <num_vms> : Number of VMs this specific host will run
- <br_ip_suffix> : Unique identifier for this host (1-254), sets bridge IP to 192.168.100.<br_ip_suffix>

**Examples:**

```
# Host 1 running 4 VMs:
bash ./script/setup_optional_cross_machine_network.sh 4 1
# Bridge IP will be: 192.168.100.1

# Host 2 running 4 VMs:
bash ./script/setup_optional_cross_machine_network.sh 4 2
# Bridge IP will be: 192.168.100.2

# Host 3 running 8 VMs:
bash ./script/setup_optional_cross_machine_network.sh 8 3
# Bridge IP will be: 192.168.100.3
```

**What this does on each host:**

- Creates VXLAN tunnel (VNI 100) for cross-host communication
- Uses multicast group 239.1.1.1 for automatic VM discovery
- Creates bridge br0 with IP 192.168.100.<br_ip_suffix>
- Creates TAP interfaces for the specified number of VMs

### 3/ Verify Cross-Host Connectivity:

```
# From Host 1, ping Host 2's bridge:ping -c 3 192.168.100.2
# From Host 1, ping Host 3's bridge:ping -c 3 192.168.100.3
# All pings should succeed
```

**Network IP Assignment**
VMs will use IPs in the 192.168.100.0/24 subnet:
• For 16 CXL Nodes: Use IPs from 192.168.100.10 to 192.168.100.25
• For 20 CXL Nodes: Use IPs from 192.168.100.10 to 192.168.100.29

## 3.2 Build QEMU Integration

The QEMU integration module must be built on each node:

```
mkdir build
cd build
cmake .. -DSERVER_MODE=ON -DCMAKE_CXX_COMPILER=g++-13
make -j$(nproc)
```

The build process uses all available CPU cores (nproc) and typically takes a few minutes depending on your hardware. The 'make install' step requires sudo privileges as it installs QEMU binaries and libraries to system directories.

## 3.3 Download and Configure VM Images

Download the pre-configured VM image and kernel. These images are provided by the OCEAN project and include necessary CXL drivers and tools:

```
gdown 1ga5CN3_H1qfReer99w_QcVOYb6R21JHI
wget https://asplos.dev/about/bzImage
```

**Note:** gdown is required to download the VM image from Google Drive. If not installed:

```
pip install gdown --break-system-packages
```

**Create VM image copies based on your deployment:**

**For Option A (Single Host running all VMs):**

If you're running 16 VMs on one host, create 16 copies:

```
cp qemu.img qemu0.img
cp qemu.img qemu1.img
cp qemu.img qemu2.img
# ... continue up to qemu15.img for 16 VMs
# OR use a loop:
for i in {0..15}; do cp qemu.img qemu$i.img; done
```

**For Option B (Multiple Hosts):**

On each host, create copies only for the VMs that host will run.

Example - Host 1 running 4 VMs:

```
cp qemu.img qemu0.img
cp qemu.img qemu1.img
cp qemu.img qemu2.img
cp qemu.img qemu3.img
```

Example - Host 2 running 4 VMs:

```
cp qemu.img qemu4.img
cp qemu.img qemu5.img
cp qemu.img qemu6.img
cp qemu.img qemu7.img
```

**Storage requirement:** Each image is approximately 30GB. Ensure you have sufficient disk space:

# 4. Fabric Manager Configuration

The Fabric Manager is the central coordinator for CXL memory pooling across the cluster. It must be started before launching any VMs.

## 4.1 Designate Primary Fabric Manager Node

Choose one node to act as the primary Fabric Manager. This should be a stable, high-availability node with good network connectivity to all other nodes.

## 4.2 Start Fabric Manager Server

On the designated Fabric Manager node, start the server. The server listens on port 9999 by default :

```
./cxlmemsim_server --capacity=1024
```

The Fabric Manager will start and listen for connections from QEMU VMs. Keep this terminal open or run the server in a screen/tmux session for persistence.

### 4.3 Verify Fabric Manager Connectivity

Before proceeding, verify that the Fabric Manager is reachable from all nodes:

```
# From each node, test connectivity
telnet <fabric_manager_node_ip> 9999
```

## 5. Virtual Machine Configuration and Launch

### 5.1 Understanding the Launch Scripts

Each host needs launch scripts for the VMs it will run. OCEAN provides two sample scripts:

- launch_qemu_cxl.sh - First VM on the host
- launch_qemu_cxl1.sh - Second VM on the host

### 5.2 Create Launch Scripts for Each VM on the Host

**If running 2 VMs on this host:** Use the provided scripts as-is

**If running more than 2 VMs:** Copy the second script to create additional scripts

```
cd /OCEAN/qemu_integration
# For 4 VMs on this host, create 2 additional scripts:
cp launch_qemu_cxl1.sh launch_qemu_cxl2.sh
cp launch_qemu_cxl1.sh launch_qemu_cxl3.sh

# For 8 VMs on this host:
for i in {2..7}; do
  cp launch_qemu_cxl1.sh launch_qemu_cxl${i}.sh
done
```

### 5.3 Customize Each Launch Script

For each VM launch script on the host, you must customize the following parameters:

### 5.3.1. TAP Network Interface

Each VM needs its own TAP interface (created by the setup_network.sh script):

```
# In launch_qemu_cxl.sh (VM 1):
-netdev tap,id=network0,ifname=tap0,script=no,downscript=no \

# In launch_qemu_cxl1.sh (VM 2):
-netdev tap,id=network0,ifname=tap1,script=no,downscript=no \

# In launch_qemu_cxl2.sh (VM 3):
-netdev tap,id=network0,ifname=tap2,script=no,downscript=no \

# Continue: tap3, tap4, tap5, etc.
```

### 5.3.2. MAC Address

Each VM must have a unique MAC address across the **entire cluster** (not just this host):

```
# In launch_qemu_cxl.sh (VM 1):
-device e1000,netdev=network0,mac=52:54:00:00:00:01 \

# In launch_qemu_cxl1.sh (VM 2):
-device e1000,netdev=network0,mac=52:54:00:00:00:02 \

# In launch_qemu_cxl2.sh (VM 3):
-device e1000,netdev=network0,mac=52:54:00:00:00:03 \

# Continue incrementing: :04, :05, :06, :07, :08, :09, :0a, :0b, :0c, :0d, :0e,
:0f, :10
```

**Important:** If you have multiple hosts, coordinate MAC addresses across all hosts. For example:

- Host 1 VMs: MAC addresses :01 through :04
- Host 2 VMs: MAC addresses :05 through :08
- Host 3 VMs: MAC addresses :09 through :0c
- And so on...

### 5.3.3. Disk Image File

Point each script to its corresponding VM image file:

```
# In launch_qemu_cxl.sh (VM 1):
-drive file=./qemu.img,index=0,media=disk,format=raw \

# In launch_qemu_cxl1.sh (VM 2):
-drive file=./qemu1.img,index=0,media=disk,format=raw \

# In launch_qemu_cxl2.sh (VM 3):
-drive file=./qemu2.img,index=0,media=disk,format=raw \

# Continue: qemu3.img, qemu4.img, qemu5.img, etc.
```

### 5.3.4. Fabric Manager Connection

All VMs must point to the Fabric Manager. Update these lines in each script:

```
# If Fabric Manager is on THIS host:
export CXL_MEMSIM_HOST=127.0.0.1
export CXL_MEMSIM_PORT=9999
export CXL_TRANSPORT_MODE=shm

# If Fabric Manager is on a DIFFERENT host:
export CXL_MEMSIM_HOST=192.168.1.5  # Replace with FM host IP
export CXL_MEMSIM_PORT=9999
export CXL_TRANSPORT_MODE=TCP
```

### 5.3.5. CXL Memory Backend Files

Each VM needs unique temporary files for CXL memory. Change these paths:

```
# In launch_qemu_cxl.sh (VM 1):
-object
memory-backend-file,id=cxl-mem1,share=on,mem-path=/tmp/cxltest1.raw,size=1G \
-object memory-backend-file,id=cxl-lsa1,share=on,mem-path=/tmp/lsa1.raw,size=1G
\




# In launch_qemu_cxl1.sh (VM 2):
-object
memory-backend-file,id=cxl-mem1,share=on,mem-path=/tmp/cxltest2.raw,size=1G \
```

```
-object memory-backend-file,id=cxl-lsa1,share=on,mem-path=/tmp/lsa2.raw,size=1G
\


# In launch_qemu_cxl2.sh (VM 3):
-object
memory-backend-file,id=cxl-mem1,share=on,mem-path=/tmp/cxltest3.raw,size=1G \
-object memory-backend-file,id=cxl-lsa1,share=on,mem-path=/tmp/lsa3.raw,size=1G
\


# Continue with unique filenames for each VM
```

## 5.4 Configure VM Network Settings and Hostnames

After creating and customizing all launch scripts, configure each VM with unique network settings:

### Step 1: Launch the first VM

```
cd /OCEAN/qemu_integration
sudo ./launch_qemu_cxl.sh
```

### Step 2: Login to the VM

- Username: root
- Password: victor129

### Step 3: Configure network and hostname inside the VM

```
# Update network configuration scripts
vi /usr/local/bin/*.sh
# Change IP address to unique IP for this VM (e.g., 192.168.100.10)


# Set hostname
vi /etc/hostname
# Change to unique hostname (e.g., cxl-vm-01)


# Shutdown to save changes
shutdown now
```

### Step 4: Repeat for all VMs on this host

Launch each VM, configure its unique IP and hostname, then shutdown:

```
# VM 2
sudo ./launch_qemu_cxl1.sh
# Inside: Set IP to 192.168.100.11, hostname to cxl-vm-02, shutdown

# VM 3
sudo ./launch_qemu_cxl2.sh
# Inside: Set IP to 192.168.100.12, hostname to cxl-vm-03, shutdown

# Continue for all VMs on this host...
```

**Coordinate IP addresses across all hosts:**

If you have multiple physical hosts, assign IP ranges to each:

- Host 1 VMs: 192.168.100.10 - 192.168.100.13 (4 VMs)
- Host 2 VMs: 192.168.100.14 - 192.168.100.17 (4 VMs)
- Host 3 VMs: 192.168.100.18 - 192.168.100.21 (4 VMs)
- And so on…

## 5.6 Launch VMs on the Host

Once all VMs are configured, launch them in stages:

**Step 1: Launch first 2-4 VMs**

```
cd /opt/OCEAN/qemu_integration
sudo ./launch_qemu_cxl.sh
sudo ./launch_qemu_cxl1.sh
sudo ./launch_qemu_cxl2.sh
sudo ./launch_qemu_cxl3.sh
```

Wait 2-3 minutes for VMs to boot.

**Stage 2: Verify they're running**

```
# Check QEMU processes
ps aux | grep qemu-system-x86_64 | grep -v grep

# Test connectivity
ping -c 3 192.168.100.10
```

**Stage 3: Launch remaining VMs**

If the first batch is successful, launch the rest:

```
sudo ./launch_qemu_cxl4.sh
sudo ./launch_qemu_cxl5.sh
# Continue for all VMs...
```

# 6. Verification and Testing

## 6.1 Verify All VMs Are Running

**Check VM count on this host:**

```
# Count running QEMU processes
ps aux | grep qemu-system-x86_64 | grep -v grep | wc -l
```

**Test VM connectivity:**

```
# Test from host
ping -c 3 192.168.100.10

# SSH into a VM
ssh root@192.168.100.10

# From inside VM, test connectivity to other VMs
ping -c 3 192.168.100.11
ping -c 3 192.168.100.15
```

## 6.2 Verify CXL Device Availability

Log into a VM and verify that CXL devices are visible:

```
# Inside VM
ls /dev/dax*
# Should show dax0.0 or similar devices
```

## 6.3 Verify Fabric Manager Logs

Check the Fabric Manager logs for any errors or warnings. The logs should show:

- Successful connection from all VMs
- Memory allocation requests and grants
- No connection timeouts or errors

# 7. Troubleshooting

## Common Issues and Solutions

| Issue | Solution |
|---|---|
| VM fails to connect to Fabric Manager | Verify Fabric Manager is running. Check firewall rules. Ensure VM launch script has correct Fabric Manager IP and port. Test network connectivity with ping and telnet. |
| CXL devices not visible in VM | Check QEMU command line includes CXL device parameters. Verify kernel module is loaded. Check dmesg for CXL-related messages. Ensure Fabric Manager allocated memory for this VM. |
| Poor performance or high latency | Check for CPU overcommitment on physical nodes. Verify network is not saturated. Review Fabric Manager logs for contention. Consider reducing number of VMs per node or upgrading network infrastructure. |
| VM crashes or becomes unresponsive | Check host system logs and QEMU logs. Monitor host memory usage. Verify sufficient resources allocated to VM. Check for kernel panics in VM serial console. |
| Network isolation between VMs | Verify bridge configuration on host nodes. Check routing tables. Ensure all VMs are on correct subnet. Verify no conflicting firewall rules. |

# 10. Appendices

## 10.1 Environment Variables Reference

| Variable | Description |
|---|---|
| `CXL_DAX_PATH` | Path to CXL DAX device, typically /dev/dax0.0 |
| `CXL_DAX_RESET` | Set to 1 to reset allocation counter on first process |
| `CXL_SHIM_VERBOSE` | Set to 1 to enable verbose logging from the CXL shim library |
| `LD_PRELOAD` | Path to CXL MPI shim library, /root/libmpi_cxl_shim.so |

## 10.2 Port and Service Reference

| Port | Service | Purpose |
|---|---|---|
| 9999 | Fabric Manager | VM connections for CXL memory management |
| 22 | SSH | VM remote access and management |

## 10.3 Additional Resources

- OCEAN GitHub Repository: https://github.com/cxl-emu/OCEAN
- CXL Emulation Workgroup Website: https://cxl-emu.github.io

# 11. Conclusion

This guide has provided comprehensive instructions for deploying the OCEAN CXL emulator across a 16-20 node cluster using the QEMU implementation. By following these steps, you have established a functional CXL memory pooling environment that enables users to benchmark applications and evaluate CXL.

Key points to remember:

- The Fabric Manager is critical and must be running before any VMs start
- Each VM requires unique IP addresses and hostnames
- Network connectivity between all nodes is essential for proper operation
- The system is designed to support various workloads including GROMACS and TIGON